

Change-Impact Reference

Every entry lists **every** file, table, column, config value and environment variable you must touch — not just the obvious one — plus what breaks if you skip a step.

Legend: ● no code change (admin panel only) · ● code change · ● code + database migration

Part 1 — Content changes

Everything in this part is ● **admin-panel only**. No files, no migrations, no deploy. The one shared caveat is caching, so it is stated once here:

Caching applies to every content change. `app/page.tsx:16`, `app/articles/page.tsx:13` and `app/articles/[slug]/page.tsx:14` all set `export const revalidate = 60`. Every save action calls `revalidatePath('/')` (via `revalidateContent()` in `app/admin/content-actions.ts`), so the change is normally immediate. If you still see stale content, hard-refresh — it clears within 60 seconds regardless.

Q: If I want to add a new Project card, what do I need to touch?

A: ● **Nothing in code.**

1. `/admin/projects` → **New project**. Fields map 1:1 to columns in `public.projects`: `title`, `category`, `year`, `summary`, `focus`, `tech` (one per line → `text[]`), `live_url`, `github_url`, `cover image`, `image_alt`, `featured`, `visible`.
2. **Cover image** uploads to Supabase Storage bucket `content-images`, under `projects/${crypto.randomUUID()}-${safeName}` — see `components/admin/ProjectForm.tsx`, `onImageSelected()`.
3. `sort_order` is assigned automatically as `max + 1` by `nextSortOrder()` in `app/admin/content-actions.ts`. Reorder with ↑ ↓.

Written by: `saveProject()` — `app/admin/content-actions.ts:337`. **Rendered by:** `components/sections/Projects.tsx` via `getProjects()` in `lib/content-db/projects.ts`.

Gotchas

- **Alt text is required whenever an image is set.** `saveProject()` returns `{ error: 'Alt text is required when there is a cover image.' }` otherwise.
- **featured changes layout, not just a badge.** Featured projects render through `FeaturedProject` (full-width, alternating sides); the rest render as grid cards via `ProjectCard`. Marking everything featured empties the grid.

- **Cover images are cropped to 16:10** (`aspect-[16/10]` + `object-cover`). A portrait photo loses its top and bottom. See Part 2.
 - **Do not edit `lib/content.ts`** to add a project. That array is the outage fallback only — it will not appear on the live site.
-

Q: If I want to add a new Skill (and a new Skill Group), what do I need to touch?

A:  **Nothing in code.**

- **A skill inside an existing group:** `/admin/skills` → Edit the group → add a line to the **Skills** textarea. One item per line; blank lines are dropped by `parseLines()` in `app/admin/content-actions.ts`. Order in the box = order of the chips.
- **A whole new group:** `/admin/skills` → **New group** → `title` + the skills list. Position it with ↑ ↓.

Table: `public.skill_groups` — `title` `text`, `skills` `text[]`. **Written by:** `saveSkillGroup()` — `content-actions.ts:278`. **Rendered by:** `components/sections/Skills.tsx` → `getSkillGroups()` in `lib/content-db/skills.ts` → `components/ui/SkillCard.tsx`.

Gotcha: the grid is `sm:grid-cols-2 lg:grid-cols-3`. Six groups fill two clean rows; seven leaves an orphan.

Q: If I want to add a new Achievement, what do I need to touch?

A:  **Nothing in code.** `/admin/achievements` → New achievement.

Table: `public.achievements` — `title`, `issuer`, `year`, `description`, `type`, `visible`, `sort_order`. **Written by:** `saveAchievement()` — `content-actions.ts:168`. **Rendered by:** `components/sections/Achievements.tsx` → `getAchievements()`.

Gotchas

- `year` is free text, not a date — "Ongoing" and "2024 – 2028" are both valid and both in use.
 - **Keep `type` short.** It renders as a badge in a `justify-between` flex row beside the year; long values overflow the card. Existing values: `Problem Solving`, `Training`, `Academic`.
 - Only `title` is required.
-

Q: If I want to change the Hero section text/stats, what do I need to touch?

A:  **Nothing in code** — but it is split across two places on one page.

`/admin/site` :

What	Where on the page	Column in <code>public.site_settings</code>
Large name heading	Identity → Full name	<code>name</code>
Line under the name	Identity → Role	<code>role</code>
Green availability pill	Identity → Availability	<code>availability</code>
"View work" button	Hero buttons → Primary button	<code>hero_cta_primary</code>
"Contact me" button	Hero buttons → Secondary button	<code>hero_cta_secondary</code>
"Résumé" link	Hero buttons → Résumé link	<code>hero_resume_label</code>
CV file the link points at	Contact details → CV / résumé path	<code>resume_url</code>
Stats (10+ Projects...)	Hero stats section (separate list)	table <code>public.hero_stats</code>
Social icons	Social links section (separate list)	table <code>public.social_links</code>

Written by: `saveSiteSettings()` (`content-actions.ts:417`) and `saveHeroStat()` (`:492`). **Rendered by:** `components/sections/Hero.tsx` → `getSiteSettings()` , `getHeroStats()` , `getSocialLinks()` .

Gotchas

- `name` is used in six places, not just the hero: the `<h1>` , the footer, the navbar initials (`initialsFrom()`), the browser tab title, the JSON-LD `Person.name` , and the About portrait's alt text. Changing it changes all six.
- **The last word of `name` gets the italic accent colour** — `Hero.tsx` splits on spaces and styles `index === nameWords.length - 1` . A one-word name means the whole thing is accented.
- `resume_url` is a path, not an upload. `/CV.pdf` maps to `public/CV.pdf` in the repo. Replacing the actual PDF is a **git commit + deploy**, not an admin change.

Q: If I want to change the About section text/facts, what do I need to touch?

A:  **Nothing in code.** `/admin/site:`

What	Column / table
"About" eyebrow	<code>site_settings.about_eyebrow</code>
Headline	<code>site_settings.about_lead</code>
Bio paragraphs	<code>site_settings.about_paragraphs</code> <code>text[]</code>
Meta row (Based in / Focus / ...)	table <code>public.about_facts</code>

Bio paragraphs are one paragraph per line. Do not double-space — blank lines are stripped by `parseLines()` , so a blank line does not create spacing, it creates nothing.

about_facts rendering rule (`components/sections/About.tsx`): `entries` with **one** item renders inline; **more than one** renders stacked lines. That is the only difference between `Based in → Cairo, Egypt` and `Focus → Full-Stack / Backend / React`.

Written by: `saveSiteSettings()`, `saveAboutFact()` (`:464`). **Not editable from the admin:** the portrait image (`public/portrait.jpeg`, referenced in `About.tsx` with `width={1122}` `height={1402}` — if you replace it, update those numbers or the layout shifts).

Q: If I want to add/remove a Social Link, what do I need to touch?

A:  **Nothing in code** — if the icon already exists. `/admin/site` → Social links → New link.

Table: `public.social_links` — `label`, `url`, `icon`, `display`, `visible`, `sort_order`.

One row appears in four places at once:

1. Hero icon row — `components/sections/Hero.tsx`
2. Footer icon row — `components/layout/Footer.tsx`
3. Contact channel list — `components/sections/Contact.tsx` (uses `display`)
4. **JSON-LD `Person.sameAs`** — `app/layout.tsx:98` (SEO)

Available icons: `github`, `linkedin`, `whatsapp`, `telegram`, `discord`, `youtube`, `mail`, `external`.

 **To add an icon that isn't on that list, three files:**

1. `components/ui/Icon.tsx` — add to the `IconName` union, add the `<path>` to `paths`, and add to the `isSolid` check if it is a filled brand mark
2. `lib/content-db/settings.ts` — add to `SOCIAL_ICONS`
3. `components/admin/SmallForms.tsx` — add to `SOCIAL_ICON_OPTIONS`

If you skip step 2, `socialIcon()` silently falls back to `external` and your glyph never renders. **If you skip step 3**, it works but you cannot select it.

Q: If I want to change Site Settings (contact info, resume label, etc.), what do I need to touch?

A:  `/admin/site`. All 23 columns of the `public.site_settings` singleton.

Fan-out worth knowing:

Column	Also affects
<code>name</code>	<code>h1</code> , footer, navbar initials, <code><title></code> , JSON-LD, portrait alt
<code>role</code>	hero subtitle, <code><title></code> , OG title

Column	Also affects
<code>description</code>	meta description, OG/Twitter description
<code>email</code>	contact row, footer link, JSON-LD, and where the contact form delivers (see Part 4)
<code>phone_tel</code>	<code>tel:</code> link + JSON-LD <code>telephone</code>
<code>phone_display</code>	visible text + the WhatsApp button's aria-label
<code>whatsapp_url</code> / <code>telegram_url</code>	blank hides that button entirely — <code>Contact.tsx</code> renders each only when set
<code>short_name</code>	navbar text label + PWA <code>short_name</code>

Cannot be changed here: `NEXT_PUBLIC_SITE_URL` (build-time env var — Part 4).

Q: If I want to add a new Service, what do I need to touch?

A:  `/admin/services` → New service.

Table: `public.services` — `title`, `description`, `deliverables text[]`, `icon`, `visible`, `sort_order`. **Written by:** `saveService()` — `content-actions.ts:246`.

Icon is a dropdown of: `code`, `layout`, `server`, `database`, `wrench`, `lifebuoy`, `gauge`, `compass`, `accessibility` — the `SERVICE_ICONS` constant in `lib/content-db/services.ts`. An unknown value falls back to `code` via `serviceIcon()`; there is **no database CHECK constraint**, so a direct SQL write can store anything.

Gotcha: the grid is `sm:grid-cols-2 lg:grid-cols-3`. Six is two clean rows; seven leaves an orphan. Cards in a row stretch to the tallest, so one service with many more deliverables adds whitespace to its neighbours.

Q: If I want to add a new Language, what do I need to touch?

A:  `/admin/skills` → Spoken languages → New language.

Table: `public.spoken_languages` — `name`, `level`. **Written by:** `saveLanguage()` — `content-actions.ts:306`. **Rendered by:** `components/sections/Skills.tsx`, below the skill grid. Both fields are required.

Q: If I want to add a new Education entry, what do I need to touch?

A:  `/admin/education` → New entry.

Table: `public.education` — `degree`, `institution`, `period`, `location`, `description`, `highlights` `text[]`. **Written by:** `saveEducation()` — `content-actions.ts:201`.

Required: `degree`, `institution`, `period`. **location** is optional and genuinely nullable — `Education.tsx` omits the whole line when it is null, which is why the DEPI entry has no location row.

Q: If I want to publish a new Article, what do I need to touch?

A: 🟢 `/admin` → **New article**. The feature already exists and is complete.

Table: `public.articles` — `title`, `slug` (**UNIQUE**), `excerpt`, `content` (Markdown), `cover_image_url`, `published`, `pinned`, `published_at`. **Written by:** `saveArticle()` — `app/admin/actions.ts:35`. **Rendered by:** `app/articles/page.tsx` and `app/articles/[slug]/page.tsx`.

Gotchas

- **Slug is auto-derived from the title** until you edit it by hand. Duplicates return *"Another article already uses that URL slug"* (Postgres 23505).
 - **Drafts are invisible to visitors even with the URL** — the RLS policy `public reads published articles` uses `using (published = true)`.
 - Images upload to bucket `article-images` (not `content-images`).
 - **Raw HTML in Markdown is escaped, not executed** — `rehype-raw` is deliberately not enabled in `components/articles/Markdown.tsx`.
 - `published_at` is stamped on first publish and **preserved** on later edits, so editing a live post does not jump it to the top.
 - **Pin/Unpin** (`/admin`) sorts an article above all others in the public list regardless of date, and shows a "Pinned" badge on both the admin row and the public card. Column `articles.pinned`; action `togglePinned()` in `app/admin/actions.ts`. Several articles can be pinned at once — they sort among themselves by date.
-

Part 2 — Structural & design changes

Q: If I want to change the order sections appear on the homepage, what do I need to touch?

A: 🟡 **Two files, and they must agree.**

1. `app/page.tsx` — reorder the JSX inside `<HomePage>`. Current order: `Hero`, `About`, `Education`, `Skills`, `Services`, `Projects`, `Achievements`, `ArticlesCta`, `Contact`.
2. `lib/site.ts` → `navLinks` — reorder to match. The comment says *"Nav order mirrors the section order on the page"* and it is not enforced.

If you skip step 2: the navbar links still work, but the order no longer matches the page, and the scroll-spy underline appears to jump around.

Do not change the `id` values (`about` , `education` , ...) — `navLinks` hrefs, the `Section` component's `aria-labelledby` , and `Hero.tsx` 's hardcoded `href="#projects"` / `href="#contact"` all depend on them.

Q: If I want to change how project cards look, what do I need to touch?

A: 🟡 **One file** — `components/sections/Projects.tsx` — but know which piece.

Sub-component	Controls
<code>ProjectImage</code> (line ~80)	The media block. <code>aspect - [16/10]</code> is what keeps every card equal height.
<code>ProjectCard</code>	The grid card
<code>FeaturedProject</code>	The full-width featured layout
<code>TechList</code> , <code>FocusBlock</code> , <code>ProjectLinks</code>	Chips, the "What it covers" block, the buttons
<code>Projects</code> (line ~186)	Section heading and the <code>mt-20 grid gap-6 lg:grid-cols-2</code> wrapper

Three rules you will break if you are not careful:

1. **Do not remove `aspect - [16/10]` from the `ProjectImage` container.** That is the entire fix for the card-height bug: without a pinned ratio the block takes the *uploaded file's* ratio, so a portrait cover produced a 612px media block against a neighbour's 307px, forcing that row 236px taller than the next.
2. **Keep `fill` on the `<Image>`.** It only works because the parent is `relative` with a fixed ratio. Swapping back to `width / height` reintroduces the bug.
3. **Keep `h-full` on `<Reveal as="li">` and on the `<article>`.** Without a definite height on the grid item, `h-full` on the article has nothing to resolve against.

To change the crop ratio, change it in **both** branches consistently, and consider `app/articles/page.tsx` (`aspect - [16/9]`) for visual consistency.

Measuring layout changes: wait for `document.images` to complete first. Measuring before images load reports the *declared* ratio, not the real one — that is how this bug was initially missed.

Q: If I want to add a new field to an existing content type, what do I need to touch?

A: 🔴 **Seven places.** Using "add `client_name` to Projects" as the worked example:

1. Migration — new file `supabase/07_project_client.sql`

```
alter table public.projects
  add column if not exists client_name text not null default '';
```

Run it **before** deploying the code. Committed, numbered, never edited after applying.

2. `lib/content-db/projects.ts` — three separate edits in one file:

- Add `client_name: string;` to `export type Project`
- Add `client_name` to the `PROJECT_COLUMNS` string
- Add `client_name: '',` to the object returned by `fallbackRows()`

3. `lib/content.ts` — add `clientName` to `export type Project` and to all three project literals (this is the compiled fallback).

4. `components/admin/ProjectForm.tsx` — add the input, `name="client_name"`, `defaultValue={project?.client_name ?? ''}`.

5. `app/admin/content-actions.ts` → `saveProject()` — read it (`const clientName = String(formData.get('client_name') ?? '').trim();`) and add it to the `values` object. **Both**, not one.

6. `components/sections/Projects.tsx` — render it.

7. `app/admin/(protected)/projects/page.tsx` — optional; show it in the list.

What breaks if you skip a step

Skipped	Symptom
1 (migration)	Could not find the 'client_name' column ... in the schema cache on every save
2 — <code>PROJECT_COLUMNS</code>	Column exists and saves, but is always undefined on the public site. Silent.
2 — <code>fallbackRows()</code>	TypeScript error; during an outage the field is missing
3	TypeScript error — <code>fallbackRows()</code> maps from these types
4	No way to enter a value
5 — reading	Field always saves as empty
5 — <code>values</code>	Reads fine, never persists
6	Stored and fetched, never displayed

The silent one is step 2. Forgetting `PROJECT_COLUMNS` produces no error anywhere — the field simply never appears.

Same seven-step shape for every type. Substitute `lib/content-db/<type>.ts`, its `*_COLUMNS` constant, `components/admin/<Type>Form.tsx`, `save<Type>()`, and `components/sections/<Type>.tsx`.

Q: If I want to change which fields are required vs optional, what do I need to touch?

A: ● Up to four layers. They are independent — changing one does not change the others.

1. **Database** — `alter table public.projects alter column category set not null;` (or `drop not null`). A new NOT NULL column also needs a default, or the ALTER fails on existing rows.
2. `app/admin/content-actions.ts` — the guard clause in the save action, e.g. `if (!title) return { error: 'Title is required.' };`. **This is the check users actually see.**
3. `components/admin/<Type>Form.tsx` — the HTML `required` attribute. Browser validation only; trivially bypassed.
4. `lib/content-db/<type>.ts` — the TypeScript type: `string` vs `string | null`.

Making something optional: relax 1, 2 and 3, then check the **rendering** component handles empty.

`Education.tsx` omits the location line when null; `Contact.tsx` hides the WhatsApp/Telegram buttons when blank. A component that assumes a value renders an empty element instead.

Making something required: backfill existing rows **first** (`update public.projects set category = 'Uncategorised' where category = ''`), or the NOT NULL constraint fails.

Q: If I want to add image upload to a section that doesn't have it yet, what do I need to touch?

A: ● Five places. Copy `components/admin/ProjectForm.tsx`, which already does this.

1. **Migration** — `add column if not exists image_url text; (nullable) and image_alt text not null default ''`;
2. `lib/content-db/<type>.ts` — add both to the type, the `*_COLUMNS` constant, and `fallbackRows()`
3. `components/admin/<Type>Form.tsx` — the upload handler. Copy `onImageSelected()` verbatim:
 - `createBrowserSupabase()` from `lib/supabase/browser.ts`
 - `const safeName = file.name.replace(/^[a-zA-Z0-9.-]/g, '-').toLowerCase()`
 - `const path = \`
`/${crypto.randomUUID()}-${safeName}``
 - `.from('content-images').upload(path, file, { cacheControl: '31536000', upsert: false })`
 - `.getPublicUrl(path)` → `useState` → `<input type="hidden" name="image_url">`
4. `app/admin/content-actions.ts` — read `image_url` / `image_alt`, store `image_url: imageUrl || null`, and require alt text when an image is set
5. `components/sections/<Type>.tsx` — render with a **pinned aspect ratio** container + `fill` + `object-cover`, and handle `null`

No `next.config.mjs` change is needed — `remotePatterns` already allows `*.supabase.co/storage/v1/object/public/**`, which covers every bucket.

No new bucket is needed — reuse `content-images` with a path prefix. A new bucket would need its own three storage policies.

Gotcha: deleting a row does **not** delete its uploaded file. There is no cleanup job; orphans stay publicly reachable.

Part 3 — Access & admin changes

Q: If I want to add a new admin user, what do I need to touch?

A:  Supabase dashboard only — and understand what you are granting.

1. Supabase → **Authentication** → **Users** → **Add user** → **Create new user**
2.  tick **Auto Confirm User** (otherwise they cannot log in)

There are no roles. Every RLS policy in this project grants write access to the `authenticated` role with `using (true)` — so **any** account that can log in has **full** control of all content. The security model depends entirely on **Authentication** → **Sign In / Providers** → **Email** → **"Allow new users to sign up" = OFF**. Verify it is still off: `POST /auth/v1/signup` must return `{"code":422,"error_code":"signup_disabled"}`.

 **For a read-only or restricted admin** you would have to rewrite every policy to check a claim, e.g. `using (auth.jwt() ->> 'role' = 'editor')`, across all 11 tables. Nothing in the current schema supports it.

Q: If I want to change what's editable from the admin panel vs. hardcoded, what do I need to touch?

A: Depends on direction.

Currently hardcoded, most likely to want moving — the five section headings:

File	Line	Value
<code>components/sections/Education.tsx</code>	13	"Where the foundations were laid"
<code>components/sections/Skills.tsx</code>	18	"The stack I reach for"
<code>components/sections/Services.tsx</code>	14	"How I can help"
<code>components/sections/Projects.tsx</code>	191	"Projects I am proud of"
<code>components/sections/Achievements.tsx</code>	14	"Milestones along the way"

(The eyebrow and description props beside them are hardcoded too. About and Contact headings **are** already database-backed.)

● To make one editable — five steps:

1. Migration: `alter table public.site_settings add column if not exists projects_title text not null default 'Projects I am proud of';`
2. `lib/content-db/settings.ts` — add to `SiteSettings` type **and** to `fallbackSettings()`
3. `app/admin/content-actions.ts` — add the name to `SETTINGS_TEXT_FIELDS`
4. `components/admin/SiteSettingsForm.tsx` — add a `<Field>`
5. `components/sections/Projects.tsx` — `await getSiteSettings()` and use it

If you skip step 3 the field renders in the form and silently never saves. **If you skip step 2** the merge in `getSiteSettings()` drops it.

Also hardcoded by necessity: `lib/site.ts` → `navLinks`, `app/sitemap.ts`, `app/robots.ts` (build-time), and `components/sections/ArticlesCta.tsx`.

Other direction — making something hardcoded again: replace the `get*()` call in the section component with a literal. Leave the table; an unused table costs nothing and losing the data is irreversible.

Q: If I want to add reorder/show-hide capability to a section that doesn't have it yet, what do I need to touch?

A: Every section already has both. If you add a **new** content type:

1. **Migration** — the table needs `visible` boolean not null default true and `sort_order` integer not null default 0, plus `create index if not exists <table>_order_idx on public.<table> (visible, sort_order);`
2. `app/admin/content-actions.ts` — add the table name to the `EDITABLE_TABLES` array. `assertTable()` throws otherwise, so `deleteRow`, `toggleVisible` and `moveRow` will all reject it.
3. **RLS** — the five standard policies (public reads visible, admin reads all, admin inserts, admin updates, admin deletes). Copy a block from `supabase/02_policies.sql`.
4. **Admin list page** — render `<RowControls table="<table>" id={row.id} title={row.title} visible={row.visible} editHref={...} isFirst={i === 0} isLast={i === rows.length - 1} />`
5. `lib/content-db/<type>.ts` — `.eq('visible', true).order('sort_order')`

If you skip step 2: the buttons render and every click throws *"Refusing to modify unknown table"*. **If you skip** `.eq('visible', true)`: hidden rows appear on the public site.

Part 4 — Infrastructure

Q: If I want to change the Supabase project, what do I need to touch?

A: 🟡 A full migration. Order matters.

1. New project + schema — run in this exact order (00 first: it defines `public.set_updated_at()`, which every later trigger needs):

```
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/00_articles.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/01_schema.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/02_policies.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/03_seed.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/04_site_schema.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/05_site_seed.sql
psql "$NEW_DB_URL" -v ON_ERROR_STOP=1 -f supabase/06_telegram.sql
```

2. Copy live data. The seed files contain the *original* content, not your edits. Use `pg_dump --data-only --table=public.<t>` per table, or export CSV from the dashboard. **Eleven tables:** `articles`, `education`, `skill_groups`, `spoken_languages`, `services`, `projects`, `achievements`, `site_settings`, `about_facts`, `hero_stats`, `social_links`.

3. Storage. Buckets `article-images` and `content-images` must be recreated (`public = true`) and **files copied manually** — they are not in any dump. **Then rewrite every stored URL**, because they embed the old project ref:

```
update public.projects
  set image_url = replace(image_url, 'OLD_REF.supabase.co', 'NEW_REF.supabase.co')
where image_url like '%OLD_REF%';
-- repeat for public.articles.cover_image_url, and for image URLs inside articles.content
```

4. Auth. Recreate your user (Add user + Auto Confirm) and **disable public sign-ups** on the new project. This is the step that, if missed, leaves your admin panel open to anyone.

5. Env vars — two places: `.env.local` and Vercel → `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Both are `NEXT_PUBLIC_*`, so **redeploy** afterwards.

No code changes. `next.config.mjs` uses `*.supabase.co`, which already covers any project ref — that wildcard exists partly for this reason.

If you forget step 3's URL rewrite: images 404 while everything else works.

Q: If I want to change the email service (away from Resend), what do I need to touch?

A: 🟡 One file, plus env vars. The integration is deliberately shallow — there is no `resend` npm package, just a `fetch`.

`app/api/contact/route.ts` — everything is here:

Line ~	What
16	<code>const RESEND_ENDPOINT = 'https://api.resend.com/emails'</code>
53	<code>FROM_ADDRESS</code>
40–46	<code>resolveToAddress()</code>
91–103	<code>escapeHtml()</code> — keep whatever provider you use
145–160	the <code>fetch</code> — headers, body, <code>AbortSignal.timeout(10_000)</code>
135–170	response handling — only returns 200 when a message <code>id</code> comes back

Env vars: remove `RESEND_API_KEY`, add the new provider's key from `.env.example`, `.env.local` and Vercel. Update `.env.example` so the contract stays documented.

Keep these behaviours — they are not Resend-specific:

- The honeypot (`company` field → silent 200)
- Validation: name, email regex, message ≥ 10 chars, max 100/200/5000
- Never returning success without provider confirmation

Nothing else imports Resend. `grep -rn "resend" app lib components` returns this one file.

Q: If I want to change the domain/deployment target, what do I need to touch?

See the dedicated answer in Part 5.

Q: If I want to add a new environment variable, what do I need to touch?

A: 🟡 Four or five places.

1. `.env.example` — add it with a comment. This is the contract for the next developer.
2. `.env.local` — the real value locally.
3. **Vercel** → **Settings** → **Environment Variables** — tick Production **and** Preview.
4. **The code that reads it.** Follow the existing pattern — **never bare** `process.env.X ?? fallback`:

```
const value = process.env.MY_VAR?.trim();
if (!value) { /* fallback */ }
```

`??` only catches `null / undefined`. An env var **set to empty string in Vercel** passes straight through — this caused a real production build failure (`TypeError: Invalid URL, input: ''`). See `resolveSiteUrl()` in `lib/site.ts` and `envOr()` in `app/api/contact/route.ts`.

5. **Redeploy if the name starts with `NEXT_PUBLIC_`**. Those are **inlined at build time**; setting them in Vercel does nothing until the next build.

Never add the Supabase `service_role` key, and never prefix a secret with `NEXT_PUBLIC_` — that ships it to every visitor's browser.

`next.config.mjs` cannot read `.env.local`. It is evaluated *before* env loading. Anything the config needs must be hardcoded — this is why `remotePatterns` uses a wildcard.

Part 5 — The two detailed questions

Q: If I want to change the email address contact-form messages are sent to, what do I need to touch — with and without Supabase?

There are **three separate addresses**. Changing the destination should not touch the other two.

Role	Value now	Where it lives
To (destination)	your inbox	env var or <code>site_settings.email</code>
From (sender)	Portfolio <onboarding@resend.dev>	<code>CONTACT_FROM_EMAIL</code> env, default at <code>route.ts:53</code>
Reply-To	the visitor's own address	<code>route.ts:153</code> , <code>reply_to: email</code> — from the form, not config

Reply-To is not a setting. It is whatever the visitor typed, so hitting Reply answers them. Changing the destination cannot break it.

The resolution order — `resolveToAddress()`, `route.ts:40`

```
const fromEnv = process.env.CONTACT_TO_EMAIL?.trim();
if (fromEnv) return fromEnv; // 1. env wins
const settings = await getSiteSettings();
return settings.email || siteConfig.email; // 2. database 3. compiled
```

Path A — without touching Supabase

Set `CONTACT_TO_EMAIL`:

1. `.env.local` → `CONTACT_TO_EMAIL=new@example.com`
2. **Vercel** → Settings → Environment Variables → same name, Production + Preview
3. **Redeploy** — required. It is server-only, but a running deployment does not pick up changed env vars.
4. Locally: **restart** `npm run dev`. Next reads `.env.local` at startup only.

This overrides the database. While it is set, editing the email in the admin changes the *displayed* address but **not** where mail goes — a genuinely confusing state. Prefer Path B unless you specifically want that split.

Path B — through Supabase (recommended)

1. Ensure `CONTACT_TO_EMAIL` is unset/empty in `.env.local` and Vercel, or Path A wins.
2. `/admin/site` → Contact details → **Email** → save. Table `public.site_settings`, column `email`. No direct SQL needed.

Caching: none for delivery. `resolveToAddress()` runs per request inside the route handler, which is dynamic — the next submission uses the new address immediately. The **displayed** address on the contact section is ISR-cached for up to 60 s, so the page may briefly show the old one while mail already goes to the new one.

Does changing the destination affect sender verification?

No. Destination and sender are independent. Receiving at `you@newdomain.com` needs nothing from Resend.

But note the current limitation: the default sender `onboarding@resend.dev` is Resend's shared address and **only delivers to the address that owns the Resend account**. If you point `to` at a different mailbox while still on the shared sender, **delivery may silently fail**. Either keep the destination as the account owner's address, or verify a domain.

If the new address also becomes the sending address

That is a separate change with DNS work:

1. Resend → **Domains** → **Add Domain** → `newdomain.com`
2. Add the records Resend gives you at your registrar — typically **MX** for the bounce subdomain, **TXT (SPF)**, **TXT (DKIM)**, and optionally **TXT (DMARC)** at `_dmarc`
3. Wait for **Verified**
4. Set `CONTACT_FROM_EMAIL="Your Name <hello@newdomain.com>"` in `.env.local` and Vercel → redeploy

Do not set `CONTACT_FROM_EMAIL` to an unverified domain — Resend rejects the send and `/api/contact` returns 502 to every visitor.

Q: If I want to change the site's domain, what do I need to touch?

1. Vercel

1. Project → **Settings** → **Domains** → **Add** → `yourdomain.com`
2. Add `www.yourdomain.com` too and let Vercel redirect one to the other
3. **Set the new domain as Production** — the `⋮` menu marks the primary

4. Keep `portfolio-ih18.vercel.app`. Do not remove it; it stays as an alias and is a useful fallback

2. DNS at your registrar

Vercel shows the exact values. Typically:

Type	Name	Value
A	@	76.76.21.21
CNAME	www	cname.vercel-dns.com

Propagation: usually minutes, up to 48 h. Vercel issues the TLS certificate automatically once DNS resolves.

Use the values Vercel displays — they change.

3. Environment variable — one name, six readers

`NEXT_PUBLIC_SITE_URL` — set it to `https://yourdomain.com` (no trailing slash; `resolveSiteUrl()` normalises anyway) in **both** `.env.local` and Vercel, then **redploy**. It is `NEXT_PUBLIC_*`, so it is **inlined at build time** — without a rebuild, every URL below keeps the old domain.

It is read in exactly one place — `lib/site.ts:22`, inside `resolveSiteUrl()`, exposed as `siteConfig.url`. **Six files consume that value:**

File	Line	Use
<code>app/layout.tsx</code>	36	<code>metadataBase</code> — the base for every relative OG/canonical URL
<code>app/layout.tsx</code>	43	<code>authors[].url</code>
<code>app/layout.tsx</code>	60	<code>openGraph.url</code>
<code>app/layout.tsx</code>	98	JSON-LD <code>Person.url</code>
<code>app/sitemap.ts</code>	7	the only sitemap entry
<code>app/robots.ts</code>	9	<code>sitemap: \${siteConfig.url}/sitemap.xml</code>
<code>app/api/contact/route.ts</code>	130	the "Sent from the contact form at ..." footer in every email

Verify after deploying: `curl https://yourdomain.com/sitemap.xml` must show the new `<loc>`. If it still says `example.com`, the variable is unset or empty — that exact failure happened before, and an *empty* value is not the same as unset.

4. Supabase

- **Authentication** → **URL Configuration** → **Site URL / Redirect URLs**. This project uses **email + password only** — no magic links, no OAuth — so nothing depends on it today. **Update it anyway:** adding any redirect-based auth later would fail confusingly.
- **CORS:** nothing to do. Supabase's REST API permits any origin; access is controlled by RLS, not origin.

- **Storage URLs:** unaffected. They point at `*.supabase.co`, not your domain, and `next.config.mjs` matches on that wildcard.

5. Resend

Only relevant if you will also **send** from the new domain — see the previous answer. Receiving is unaffected. Changing your site's domain does **not** invalidate any existing Resend setup.

6. Hardcoded references to hunt down

```
grep -rn "portfolio-ih18\|vercel\.app" \
  --include="*.ts" --include="*.tsx" --include="*.md" --include="*.json" \
  app components lib supabase *.md
```

Known occurrences at time of writing:

File	What
<code>HANDOFF.md</code> / <code>HANDOFF.pdf</code>	Header + quick-reference card
<code>TECHNICAL.md</code> / <code>TECHNICAL.pdf</code>	Header
<code>CHANGES.md</code>	This file
<code>README.md</code>	Check — may reference the URL
<code>lib/content.ts</code>	<code>https://frontend-playground-ih18.vercel.app</code> and the two <code>depiassignment*.vercel.app</code> URLs are your project demos, not this site — leave them alone
Database <code>public.projects.live_url</code>	Same: other projects' URLs

No source file hardcodes this site's own domain — everything goes through `siteConfig.url`. Only the documentation needs updating.

7. SEO

- **Keep the old domain permanently redirecting.** Vercel issues a **308** by default (a permanent redirect, and search engines treat it like a 301) once the new domain is primary and the old is still attached. Do not delete the old domain — that produces 404s and drops any accumulated ranking.
- `alternates: { canonical: '/' }` in `app/layout.tsx:56` resolves against `metadataBase`, so canonicals follow automatically once the env var is set and you have redeployed.
- Resubmit `https://yourdomain.com/sitemap.xml` in Google Search Console and use the **Change of Address** tool if the old domain was verified there.
- `app/robots.ts` disallows `/admin` and points at the sitemap — both follow the env var, nothing to edit.

The single most common mistake: adding the domain in Vercel but forgetting to update

`NEXT_PUBLIC_SITE_URL` and redeploy. The site loads perfectly on the new domain while every canonical tag, OG URL and the sitemap still advertise the old one — which actively harms SEO and is invisible in the browser.